

“그리드 복원 (grid_encoding)” 문제 풀이

작성자: 조승현, 박상훈

부분문제 1

$A_{i,i}$ 와 $A_{i,i+1}$ 은 모두 1이며, 이 셀들을 모두 select하면 $2N - 1$ 개의 셀이 선택된다. 한편, 0부터 $2N - 1$ 까지 $2N$ 개의 정점이 있는 그래프 G 에서 셀 (i, j) 를 정점 i 와 $N + j$ 사이에 간선을 잇는다고 생각하면 선택한 셀들로 만든 그래프 G 는 모든 정점을 연결하는 길이 $2N - 1$ 의 path가 되고, $N - 1$ 번 및 N 번 정점이 path의 양 끝점이 된다. B 에서도 값이 1인 셀들에 대해 그래프를 만들면 동일하게 길이 $2N - 1$ 의 path가 되고, 이를 통해 행과 열의 순서가 어떻게 바뀌었는지, 즉 순열 X 와 Y 를 알 수 있어 문제를 해결할 수 있다.

부분문제 2

함수 `reconstruct`의 파라미터는 행과 열이 순열 X, Y 에 의해 바뀐 후 주어지므로, 행과 열을 적절히 재배열하여도 문제가 없어야 한다. 오른쪽(아래쪽)으로 갈수록 열(행)에 있는 검은 칸 개수가 단조증가하게 재배열해놓고 생각하자. 왼쪽 아래 꼭짓점에서 시작해서 오른쪽 위 꼭짓점까지 흰색 칸과 검은색 칸의 경계를 그릴 수 있다. 경계 왼쪽 위 영역은 흰색, 오른쪽 아래 영역은 모두 검은색이다.

$K \leq 4N$ 풀이: 경계와 인접한 모든 칸을 선택한다. 다른 색이 존재하는 행/열의 경우, 다른 색이 각각 1개 이상 선택되어있다. 즉, 선택된 칸의 색이 모두 같은 행/열은 나머지 원소도 색이 모두 같다. 복원을 하기 위해서는 다음 과정을 반복한다. 선택된 칸의 색이 모두 같은 행/열 중 선택된 칸의 개수가 최소인 행/열을 아무거나 하나 선택하고 같은 색으로 칠한다. 색을 칠하고 나면 그 행/열은 더 이상 고려할 필요가 없기 때문에 격자에서 제거하고 재귀적으로 문제를 해결한다. 이 과정을 반복할 때 격자는 앞에서 언급한 성질을 계속 만족한다는 것을 알 수 있다.

$K \leq 2N - 1$ 풀이: 경계의 가로선 바로 위에 있는 흰색 칸, 세로선 바로 오른쪽에 있는 검은색 칸을 선택한다. 인접하는 칸 (행/열을 공유하는 칸)끼리 연결하면 체인을 구성할 수 있고 이를 통해 답을 복원할 수 있다.

부분문제 3

각 행에서 검은색인 열들의 집합은 서로 포함관계를 이룬다. 이에 따라 행과 열을 적절히 재배열하여 히스토그램 형태로 만들 수 있으므로 부분문제 2와 동일하게 해결할 수 있다.

풀이 2. 부분문제 1과 같이 그래프 G 를 만드는데, 이번에는 방향성 간선으로 만든다. $A_{i,j} = 1$ 이면 i 에서 $N + j$ 로, $A_{i,j} = 0$ 이면 $N + j$ 에서 i 로 간선을 만든다. 이 때 사이클이 존재하지 않음을 증명할 수 있다.

얻은 DAG는 (행들) (열들) (행들) (열들) ... 과 같은 위상정렬 순서를 가지게 되고, 각 그룹 내에서의 순서는 바뀔 수 있지만 다른 그룹사이의 순서는 바뀔 수 없다.

각 정점 u 에 대해 자기보다 뒤에 나오는 그룹 중 가장 앞 원소 v 가 존재한다면 간선 (u, v) 에 해당하는 셀을 select하자.

그러면 그 셀들을 보고 위상정렬 순서를 완전히 reconstruct할 수 있다. 따라서, 원래 그리드를 복원할 수 있다.

“입자 가속기 (particles)” 문제 풀이

작성자: 박상훈

부분문제 1

IOI 입자 생성에 실패한 방은 트리에서 지워버린다고 생각해도 무방하다. 즉, 포레스트에서 문제를 해결한다고 생각한다. 각 컴포넌트에서 할 수 있는 충돌 실험의 횟수는 IOI 입자의 개수의 절반을 넘지 않는 최대 정수다. 매 실험마다 IOI 입자가 2개씩 사라지므로 이보다 충돌 실험을 많이 할 수 없다. 충돌 실험에 쓰일 두 입자를 아무거나 고른 후, 그 둘 사이에 다른 입자가 있으면 둘 중 하나를 그 입자로 교체하는 것을 반복하면 실패를 구성할 수 있다. 따라서, 매 쿼리마다 Tree DP로 각 컴포넌트에 있는 IOI 입자의 개수를 $O(N)$ 시간에 세면 문제를 해결할 수 있다.

부분문제 2

세그먼트 트리로 문제를 해결한다. l 이상 r 이하 구간을 나타내는 노드에 대해, l 이상 r 이하의 방만 존재한다고 생각하고 다음 4가지 값을 저장한다: 충돌 실험 횟수의 최댓값, 가장 왼쪽 컴포넌트에 있는 IOI 입자 개수, 가장 오른쪽 컴포넌트에 있는 IOI 입자 개수, IOI 입자 생성에 실패한 방의 개수. 케이스워크를 통해 세그먼트 트리에서 왼쪽 자식 노드와 오른쪽 자식 노드를 $O(1)$ 시간에 합칠 수 있다. 따라서, 문제를 쿼리당 $O(\log N)$ 시간에 해결할 수 있다.

부분문제 3

각 컴포넌트에 속하는 정점을 명시적으로 관리하는 방식으로 해결할 수 있다. IOI 입자 생성에 실패할 경우, 새로 생기는 컴포넌트를 DFS로 $O(N)$ 시간에 구해줄 수 있다. 각 정점이 어떤 컴포넌트에 속하는지 저장해두면 IOI 입자 생성에 성공했을 때 $O(1)$ 시간에 답을 갱신할 수 있다. IOI 입자 생성에 실패하는 경우가 적기 때문에 부분문제 3을 해결할 수 있다.

부분문제 4

어떤 두 IOI 입자를 선택해도 그 경로 위에 존재하지 않는 방이 있다면, 그 방은 IOI 입자 생성에 실패해도 답에 영향을 주지 않는다. DFS를 통해 이러한 방을 $O(N)$ 시간에 모두 찾아줄 수 있다. IOI 입자 생성에 성공하거나, 답에 영향을 줄 수도 있는 방에서 IOI 입자 생성에 실패했을 경우, DFS를 다시 해서 정보를 갱신한다. DFS를 할 때마다 IOI 입자 개수가 증가하거나, IOI 입자가 있는 컴포넌트 개수가 증가하므로, DFS를 하는 횟수는 IOI 입자 개수의 최댓값의 두 배 이하이다. 따라서, 부분문제 4를 시간제한 내에 해결할 수 있다.

부분문제 5

트리의 루트를 0으로 설정한다. DFS ordering으로 N 개의 정점의 in 과 out 을 모두 기록한 길이 $2N$ 의 배열을 계산한다. 쿼리를 처리할 때마다 다음 성질을 만족하도록 길이 $2N$ 의 배열 A 를 관리한다: 컴포넌트의 루트 v 에 대해, $\sum_{i=in[v]}^{out[v]} A[i] = 0$, $out[v] = -(\text{컴포넌트에 있는 IOI 입자의 개수})$. 이때, 컴포넌트의 루트는 깊이가 최소인 유일한 정점으로 정의한다. 어떤 정점이 속하는 컴포넌트의 루트는 구간 업데이트를 지원하는 세그먼트 트리를 이용하면 $O(\log N)$ 시간에 갱신, 계산 가능하다. 방 x 에서 IOI 입자 생성에 성공했을 경우, $A[in[x]]$ 에 1을 더하고 $A[out[root]]$ 에 -1 을 더하면 배열 A 의 조건을 만족하면서 A 를 갱신할 수 있다. 방 y 에서 IOI 입자 생성에 실패했을 경우, y 의 자식을 보면서 각각 새로운 컴포넌트의 루트로 설정해준다. y 의

자식 z 에 대해, z 가 속하게 되는 새로운 컴포넌트에 있는 IOI 입자는 $\sum_{i=in[z]}^{out[z]} A[i]$ 가 된다. z 의 서브트리 구간에 속하는 다른 컴포넌트에 대해 A 의 구간합은 0이기 때문이다. 따라서, y 의 차수를 d 라고 할 때 $O(d \log N)$ 시간에 갱신할 수 있다. 따라서, 전체 시간복잡도는 $O(N + Q \log N)$ 이 된다.

“다리 보수 공사 (roadwork)” 문제 풀이

작성자: 구재현

부분문제 1

가능한 입력은 "AB", "BA" 두 가지 뿐이다. 두 경우 모두 답은 [2, 1] 이다.

부분문제 2

모든 가능한 2^{2N-2} 가지의 다리 부분집합을 완전 탐색하여 해결 가능하다.

부분문제 5

동적 계획법을 사용한다. $DP[i]$ 를, $0, \dots, i$ 번 다리까지의 공사 여부를 고정했고, 이 중 i 번 다리를 공사할 때, 가능한 최대 개수와 그 경우의 수라고 정의하자. i 번 다리와 $j (< i)$ 번 다리를 같이 공사하기 위해서는, 두 다리가 같은 정점을 공유하여서는 안되며, 이는 두 다리 사이에 문자 'A', 'B' 가 모두 등장해야 한다는 것이다. 즉, $S[j], S[j+1], \dots, S[i-1]$ 중에 'A', 'B' 가 모두 등장할 경우, j 에서 i 로 상태 전이가 가능하다. 각 i, j 에 대해 상태 전이 가능 여부를 $O(N)$ 에 판별할 수 있으니, 시간 복잡도는 $O(N^3)$ 이다.

부분문제 7

부분문제 5의 풀이를 약간 개선한다. i 를 고정시키고 j 를 감소시키면서 전이를 고려할 경우, 상태 전이 가능 여부를 매번 따로 계산할 필요가 없다. 시간 복잡도는 $O(N^2)$ 이다.

부분문제 10

(X_i, Y_i) 를 $S[0], \dots, S[i-1]$ 중 'A', 'B'의 등장 횟수로 정의하자. 문제에서 요구하는 것은, X 좌표와 Y 좌표가 모두 증가하는 점들의 최대 크기 부분집합과 그 개수다. 이는 LIS를 구하는 것과 동일하게 세그먼트 트리를 사용하여 $O(N \log N)$ 에 계산할 수 있다.

이분 탐색을 통하여 LIS를 구하는 풀이를 사용하면 부분 점수를 얻을 수 있다.

부분문제 11

$f(i)$ 를, $S[j], S[j+1], \dots, S[i-1]$ 중에 'A', 'B' 가 모두 등장하는 최대 $j < i$ 라고 정의하자. 모든 i 에 대해 $f(i)$ 는 Two pointers를 사용하여 $O(N)$ 시간에 계산할 수 있다. 이렇게 할 경우, j 에서 i 로 상태 전이가 가능하다는 것은 $j \leq f(i)$ 임과 동치다. 모든 $j \leq f(i)$ 에 대해서, $DP[j]$ 의 최댓값과, 최댓값을 주는 경우 그 경우의 수의 합을 빠르게 계산하여야 한다. 이는 DP 값을 계산함과 동시에 부분합 배열을 만드는 식으로 $O(1)$ 에 계산할 수 있다. 시간 복잡도는 $O(N)$ 이다.

최댓값만 계산하는 경우, Two pointers와 DP 없이 단순한 그리디 알고리즘을 사용할 수 있다. 이를 통해 부분 점수를 얻을 수 있다.

“안전 지대 (safezone)” 문제 풀이

작성자: 박상훈, 구재현

부분문제 1

$B[i] \leq 0 \leq D[i]$ 조건에 따라서, 직사각형을 $[A[i], C[i]]$ 구간의 점을 포함하는 수직선 상 선분으로 생각할 수 있다.

만약에 i 번 선분과 $j(> i)$ 번 선분이 같은 연합에 속한다면, $i, i+1, \dots, j-1, j$ 번 선분이 모두 같은 연합에 속함을 알 수 있다. 각 $0 \leq i \leq N-2$ 에 대해서, i 번 선분과 $i+1$ 번 선분이 같은 연합에 속한다는 것은, $\max_{j \leq i} C[j] \geq \min_{j > i} A[j]$ 를 만족한다는 것과 동치이다. Prefix/Suffix Minimum을 취하여 이를 판단할 수 있고, 이 값을 토대로 각 선분이 속하는 연합을 구할 수 있다.

부분문제 2

선분을 $A[i]$ 가 증가하는 순으로 정렬하면, 부분문제 1과 같은 풀이가 성립한다.

부분문제 3

$O(N^2)$ 시간에 그래프를 구성한 후 연결 컴포넌트를 계산하여 해결할 수 있다.

부분문제 4

$A[i] = C[i]$ 인 직사각형을 세로 선분, $B[i] = D[i]$ 인 선분을 가로 선분 이라고 부르자. 또한, $O(N \log N)$ 시간에 모든 직사각형 좌표를 $[0, 2N-1]$ 범위로 줄여주자 (좌표 압축).

x 좌표가 증가하는 방향으로 스윕한다. 가로 선분은 $x = A[i]$ 시점에 삽입, $x = C[i]+1$ 시점에 삭제되며, 세로 선분은 $x = A[i]$ 시점에 처리된다. 세로 선분이 들어올 때, 현재 자료구조에 있는 가로 선분 중 y 좌표가 $[B[i], D[i]]$ 사이에 있는 선분들을 모두 모아서, 이들을 같은 연결 컴포넌트에 넣어주어야 한다.

고로, 다음 연산을 빠르게 하는 자료구조 T 가 필요하다:

- $\text{UPDATE}(i, x)$: $T[i] = x$ 로 값 설정.
- $\text{MERGE}(l, r, x)$: $T[l], T[l+1], \dots, T[r]$ 중 -1 이 아닌 모든 $T[i]$ 에 대해, $T[i]$ 와 x 간에 간선 추가 (혹은 Union-Find에서 같은 정점으로 합쳐줌)

이는 세그먼트 트리를 사용하여 해결할 수 있다. 세그먼트 트리의 각 노드에 대해서 다음 세 값을 관리하자:

- C : 해당 구간 안에서 $T[i] \neq -1$ 인 i 의 수
- L : 해당 구간에 있는 모든 $T[i] \neq -1$ 에 대해서, 간선을 추가해 줘야 할 컴포넌트 (없으면 -1)
- V : (리프 노드의 경우) $T[i]$

C, V 는 어렵지 않게 관리할 수 있다. L 은 Lazy Propagation을 사용하여 관리한다. 부모 구간에 있는 L 을 자식 구간으로 전파시키는 연산을 다음과 같이 처리한다: 자식이 $C = 0$ 일 경우 무시하고, 자식 구간의 L 이 -1 이 아닐 경우 자식 구간과 부모 구간 간에 간선을 추가하자. 이제, 자식 구간의 L 은 -1 이거나 이미 부모와 연결되어 있으니 부모의 L 을 덮어쓰면 된다. 이렇게 할 경우 어떠한 노드의 L 을 자식으로

전파시켜줄 수 있다. 이를 사용하여 MERGE 연산은 $O(\log N)$ 개의 구간에만 적용해 주고, UPDATE 연산을 수행할 때도 $O(\log N)$ 개의 구간에 대해서 전파를 수행하여 $L = -1$ 인 상황을 만들어 줄 수 있다.

시간 복잡도는, 간선 추가가 $O(1)$ 에 된다고 가정하였을 때 $O(N \log N)$ 이다.

이 풀이에 분할 정복을 추가하여 전체 문제를 $O(N \log N)$ 에 해결하는 별해가 존재한다.

부분문제 5

$O(N^2)$ 보다 효율적이지만 의도된 풀이만큼 빠르지는 않은 풀이들은 부분문제 5를 맞을 수 있다. 출제진이 작성한 모든 $O(N\sqrt{N})$ 풀이와 일부 $O(N \log^2 N)$ 풀이가 이에 해당한다.

부분문제 6

x 좌표가 증가하는 방향으로 스윕한다. y 좌표를 인덱스로 하는 세그먼트 트리를 관리하면 현재 x 좌표에서 특정 y 좌표 구간에 직사각형이 존재하는지 $O(\log N)$ 에 판별할 수 있다. y 좌표 구간 $[l, r]$ 에 직사각형이 있다면, i 번 직사각형과 같은 컴포넌트라는 것을 의미하는 (l, r, i) 튜플을 관리하는 자료구조를 생각하자. 직사각형을 삭제할 때, 실제로 직사각형이 존재하는 구간과 일대일 대응되도록 튜플을 저장하면 최악의 경우 튜플의 개수가 $O(N)$ 씩 변하게 되어 전체 시간복잡도가 $O(N^2)$ 이상이 된다. 그러나, 직사각형이 삭제되어도 저장되어 있는 튜플은 여전히 정의를 만족하므로 정보를 갱신할 필요가 없다. 따라서, 직사각형을 새로 삽입하는 경우만 추가로 관리해주면 된다. 직사각형을 새로 삽입할 때 교집합이 존재하는 튜플을 모두 순회하며 새로운 튜플 하나로 합쳐주게 되면 삽입/삭제는 $O(N)$ 번 발생하게 된다. 이때, 튜플로 표현된 구간에 직사각형이 실제로 존재하는지 확인하기 위해 앞에서 언급한 세그먼트 트리를 사용해야 하며, Union-Find로 컴포넌트를 관리하면 된다. 튜플을 저장하는 자료구조는 `std::set`을 사용하면 된다. 시간복잡도는 $O(N \log N)$ 이다.